

Fix C++26 by making the symmetric and Hermitian rank-k and rank-2k updates consistent with the BLAS

Document #: P3371R0
Date: 2024-08-07
Project: Programming Language C++
LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- [1 Authors](#)
- [2 Revision history](#)
- [3 Abstract](#)
- [4 Motivation](#)
- [5 Proposed changes](#)
- [6 We do NOT propose changing rank-1 or rank-2 update functions](#)
- [7 Fixes for some wording issues](#)
- [8 Wording](#)
 - [8.1 New exposition-only concept](#)
 - [8.2 Rank-k update functions in synopsis](#)
 - [8.3 Rank-2k update functions in synopsis](#)
 - [8.4 Specification of rank-k update functions](#)
 - [8.5 Specification of rank-2k update functions](#)

§ 1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

§ 2 Revision history

- Revision 0 to be submitted 2024-08-15

§ 3 Abstract

The four `std::linalg` functions `symmetric_matrix_rank_k_update`, `hermitian_matrix_rank_k_update`, `symmetric_matrix_rank_2k_update`, and `hermitian_matrix_rank_2k_update` currently have behavior inconsistent with their corresponding BLAS (Basic Linear Algebra Subroutines) routines. Also, their behavior is inconsistent with `std::linalg`'s `matrix_product`, even though in mathematical terms they are special cases of a matrix-matrix product.

We propose fixing this by making two changes. First, we will add “updating” overloads analogous to the updating overloads of `matrix_product`. For example, `symmetric_matrix_rank_k_update(A, scaled(beta, C), C, upper_triangle)` will perform $C := \beta C + AA^T$. Second, we will change the behavior of the existing overloads to be “overwriting,” without changing the number of their parameters or how they are called. For example, `symmetric_matrix_rank_k_update(A, C, upper_triangle)` will perform $C := AA^T$ instead of $C := C + AA^T$.

The second set of changes is a breaking change. Thus, we must finish this before finalization of C++26.

§ 4 Motivation

Each function in `std::linalg` generally corresponds to one or more routines or functions in the original BLAS (Basic Linear Algebra Subroutines). Every computation that the BLAS can do, a function in `std::linalg` should be able to do.

One `std::linalg` user [reported](#) an exception to this rule. The BLAS routine `DSYRK` (Double-precision SYmmetric Rank-K update) computes $C := \beta C + \alpha AA^T$, but the corresponding `std::linalg` function `symmetric_matrix_rank_k_update` only computes $C := C + \alpha AA^T$. That is, `std::linalg` currently has no way to express this BLAS operation with a general β scaling factor. This issue applies to all of the following functions.

- `symmetric_matrix_rank_k_update`: computes $C := C + \alpha AA^T$
- `hermitian_matrix_rank_k_update`: computes $C := C + \alpha AA^H$
- `symmetric_matrix_rank_2k_update`: computes $C := C + \alpha AB^H + \alpha BA^H$
- `hermitian_matrix_rank_2k_update`: computes $C := C + \alpha AB^H + \bar{\alpha} BA^H$, where $\bar{\alpha}$ denotes the complex conjugate of α

These functions implement special cases of matrix-matrix products. The `matrix_product` function in `std::linalg` implements the general case of matrix-matrix products. This function corresponds to the BLAS's `SGEMM`, `DGEMM`, `CGEMM`, and `ZGEMM`, which compute $C := \beta C + \alpha AB$, where α and β are scaling factors. The `matrix_product` function has two kinds of overloads:

1. *overwriting* ($C = AB$) and

2. *updating* ($C = E + AB$).

The updating overloads handle the general α and β case by `matrix_product(scaled(alpha, A), B, scaled(beta, C), C)`. The specification explicitly permits the input `scaled(beta, C)` to alias the output `C` ([`linalg.blas3.gemm`] 10: “Remarks: `C` may alias `E`”). The `std::linalg` library provides overwriting and updating overloads so that it can do everything that the BLAS does, just in a more idiomatically C++ way. Please see [P1673R13 Section 10.3](#) (“Function argument aliasing and zero scalar multipliers”) for a more detailed explanation.

The problem with functions like `symmetric_matrix_rank_k_update` is that they have the same *calling syntax* as the overwriting version of `matrix_product`, but *semantics* that differ from both the overwriting and the updating versions of `matrix_product`. The current overloads are not overwriting, so we can’t just fix this problem by introducing an “updating” overload for each function.

Incidentally, the fact that these functions have “update” in their name is not relevant, because that naming choice is original to the BLAS. The BLAS calls its corresponding `xSYRK`, `xHERK`, `xSYR2K`, and `xHER2K` routines “{Symmetric, Hermitian} rank {one, two} update,” even though setting $\beta = 0$ makes these routines “overwriting” in the sense of `std::linalg`.

§ 5 Proposed changes

We propose to fix this by making the four functions work just like `matrix_vector_product` or `matrix_product`. This entails two changes.

1. We will add “updating” overloads with a new input matrix parameter `E`, analogous to the updating overloads of `matrix_product`. For example, `symmetric_matrix_rank_k_update(A, E, C, upper_triangle)` will compute $C = E + AA^T$. We will specifically permit `C` and `E` to alias, thus permitting the desired case where `E` is `scaled(beta, C)`. The updating overloads will take `E` as an *in-matrix*, and will change `C` from a *possibly-packed-inout-matrix* to a *possibly-packed-out-matrix*.
2. We will change the behavior of the existing overloads to be “overwriting,” without changing how they are called. For example, `symmetric_matrix_rank_k_update(A, C, upper_triangle)` will compute $C = AA^T$ instead of $C := C + AA^T$. The overwriting overloads will change `C` from a *possibly-packed-inout-matrix* to a *possibly-packed-out-matrix*.

This second set of changes is a breaking change. It needs to be so that we can provide the overwriting behavior $C := \alpha AA^T$ that the corresponding BLAS routines already provide. Thus, we must finish this before finalization of C++26.

Both sets of overloads still only write to the specified triangle (lower or upper) of the output matrix C. As a result, the new updating overloads only read from that triangle of the input matrix E. Therefore, even though E may be a different matrix than C, the updating overloads do not need an additional `Triangle t_E` parameter for E. The `symmetric_*` functions interpret E as symmetric in the same way that they interpret C as symmetric, and the `hermitian_*` functions interpret E as Hermitian in the same way that they interpret C as Hermitian.

§ 6 We do NOT propose changing rank-1 or rank-2 update functions

The four functions we propose to change have the following rank-1 and rank-2 analogs, where A denotes a symmetric or Hermitian matrix (depending on the function's name) and x and y denote vectors.

- `symmetric_matrix_rank_1_update`: computes $A := A + \alpha x x^T$
- `hermitian_matrix_rank_1_update`: computes $A := A + \alpha x x^H$
- `symmetric_matrix_rank_2_update`: computes $A := A + \alpha x y^T + \alpha y x^T$
- `hermitian_matrix_rank_2_update`: computes $A := A + \alpha x y^H + \bar{\alpha} y x^H$

We do NOT propose to change these functions analogously to the rank-k and rank-2k update functions. This is because the BLAS routines corresponding to the rank-1 and rank-2 functions – `xSYR`, `xHER`, `xSYR2`, and `xHER2` – do not have a way to supply a β scaling factor. That is, these `std::linalg` functions can already do everything that their corresponding BLAS routines can do. This is consistent with our design intent in [Section 10.3 of P1673R3](#) for translating Fortran `INTENT(INOUT)` arguments into a C++ idiom.

- b. Else, if the BLAS function unconditionally updates (like `xGER`), we retain read-and-write behavior for that argument.
- c. Else, if the BLAS function uses a scalar beta argument to decide whether to read the output argument as well as write to it (like `xGEMM`), we provide two versions: a write-only [that is, “overwriting”] version (as if beta is zero), and a read-and-write [that is, “updating”] version (as if beta is nonzero).

The rank-1 and rank-2 update functions “unconditionally update,” in the same way that the BLAS’s general rank-1 update routine `xGER` does. However, the BLAS’s rank-k and rank-2k update functions “use a scalar beta argument...,” so for consistency, it makes sense for `std::linalg` to provide both overwriting and updating versions.

Since we do not propose changing the symmetric and Hermitian rank-1 and rank-2 functions, we retain the exposition-only concept *possibly-packed-inout-matrix*, which they use to constrain their parameter A.

§ 7 Fixes for some wording issues

There are some wording issues in `[linalg.blas3.rankk]` and `[linalg.blas3.rank2k]`, specifically in the Mandates, Preconditions, and Complexity clauses. These clauses do not reflect the design intent, which is expressed by the corresponding BLAS routines and the mathematical expressions in the current wording that express the functions' behavior. We have not yet filed an LWG issue for these wording issues. Given that the above changes will call for adjusting some parts of these clauses anyway (e.g., by changing `InOutMat` to `OutMat`), we have decided to propose “drive-by” fixes to these clauses in this proposal. We nevertheless plan to file an LWG issue to fix both these and other wording issues we have found.

Many thanks (with permission) to Raffaele Solcà (CSCS Swiss National Supercomputing Centre, raffaele.solca@cscs.ch) for pointing out this issue and related issues that we plan to file as part of the aforementioned LWG issue.

§ 8 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The  character is used to denote a placeholder section number which the editor shall determine.

In `[version.syn]`, for the following definition,

```
#define __cpp_lib_linalg YYYYMML // also in <linalg>
```

adjust the placeholder value `YYYYMML` as needed so as to denote this proposal's date of adoption.

§ 8.1 New exposition-only concept

In the header `<linalg>` synopsis `[linalg.syn]`, immediately before the following,

```
template<class T>
    concept possibly-packed-inout-matrix = see below; //
exposition only
```

add the following.

```
template<class T>
    concept possibly-packed-out-matrix = see below; //
exposition only
```

Then, to `[linalg.helpers.concepts]`, immediately before the following,

```
template<class T>
    concept possibly-packed-inout-matrix =
```

```

        is-mdspan<T> && T::rank() == 2 &&
        is_assignable_v<typename T::reference, typename
T::element_type> &&
        (T::is_always_unique() || is-layout-blas-packed<typename
T::layout_type>);

```

add the following definition of the exposition-only concept *possibly-packed-out-matrix*.

```

template<class T>
concept possibly-packed-out-matrix =
    is-mdspan<T> && T::rank() == 2 &&
    is_assignable_v<typename T::reference, typename
T::element_type> &&
    (T::is_always_unique() || is-layout-blas-packed<typename
T::layout_type>);

```

§ 8.2 Rank-k update functions in synopsis

In the header `<linalg>` synopsis [`linalg.syn`], change the following

```

// rank-k symmetric matrix update
template<class Scalar, in-matrix InMat, possibly-packed-inout-
matrix InOutMat, class Triangle>
    void symmetric_matrix_rank_k_update(Scalar alpha, InMat A,
InOutMat C, Triangle t);
template<class ExecutionPolicy, class Scalar,
        in-matrix InMat, possibly-packed-inout-matrix InOutMat,
class Triangle>
    void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec,
        Scalar alpha, InMat A,
InOutMat C, Triangle t);

template<in-matrix InMat, possibly-packed-inout-matrix InOutMat,
class Triangle>
    void symmetric_matrix_rank_k_update(InMat A, InOutMat C,
Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat, possibly-packed-inout-matrix InOutMat,
class Triangle>
    void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec,
        InMat A, InOutMat C,
Triangle t);

// rank-k Hermitian matrix update
template<class Scalar, in-matrix InMat, possibly-packed-inout-
matrix InOutMat, class Triangle>
    void hermitian_matrix_rank_k_update(Scalar alpha, InMat A,
InOutMat C, Triangle t);
template<class ExecutionPolicy,
        class Scalar, in-matrix InMat, possibly-packed-inout-
matrix InOutMat, class Triangle>
    void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec,
        Scalar alpha, InMat A,

```

```

InOutMat C, Triangle t);

    template<in-matrix InMat, possibly-packed-inout-matrix InOutMat,
class Triangle>
        void hermitian_matrix_rank_k_update(InMat A, InOutMat C,
Triangle t);
    template<class ExecutionPolicy,
            in-matrix InMat, possibly-packed-inout-matrix InOutMat,
class Triangle>
        void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec,
                                            InMat A, InOutMat C,
Triangle t);

```

to read as follows.

```

// overwriting rank-k symmetric matrix update
template<class Scalar,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
    void symmetric_matrix_rank_k_update(
        Scalar alpha, InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy, class Scalar,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
    void symmetric_matrix_rank_k_update(
        ExecutionPolicy&& exec, Scalar alpha, InMat A, OutMat C,
Triangle t);
template<in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
    void symmetric_matrix_rank_k_update(
        InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
    void symmetric_matrix_rank_k_update(
        ExecutionPolicy&& exec, InMat A, OutMat C, Triangle t);

// updating rank-k symmetric matrix update
template<class Scalar,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
    void symmetric_matrix_rank_k_update(
        Scalar alpha,
        InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy, class Scalar,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>

```

```

        void symmetric_matrix_rank_k_update(
            ExecutionPolicy&& exec, Scalar alpha,
            InMat1 A, InMat2 E, OutMat C, Triangle t);
template<in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
        void symmetric_matrix_rank_k_update(
            InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
        void symmetric_matrix_rank_k_update(
            ExecutionPolicy&& exec,
            InMat1 A, InMat2 E, OutMat C, Triangle t);

// overwriting rank-k Hermitian matrix update
template<class Scalar,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
        void hermitian_matrix_rank_k_update(
            Scalar alpha, InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy, class Scalar,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
        void hermitian_matrix_rank_k_update(
            ExecutionPolicy&& exec, Scalar alpha, InMat A, OutMat C,
Triangle t);
template<in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
        void hermitian_matrix_rank_k_update(
            InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat,
        class Triangle>
        void hermitian_matrix_rank_k_update(
            ExecutionPolicy&& exec, InMat A, OutMat C, Triangle t);

// updating rank-k Hermitian matrix update
template<class Scalar,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
        void hermitian_matrix_rank_k_update(
            Scalar alpha,
            InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy, class Scalar,
        in-matrix InMat1,

```



```

        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec, Scalar alpha,
    InMat1 A, InMat2 E, OutMat C, Triangle t);
template<in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_k_update(
    InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    InMat1 A, InMat2 E, OutMat C, Triangle t);

```

§ 8.3 Rank-2k update functions in synopsis

In the header `<linalg>` synopsis [[linalg.syn](#)], change the following

```

// rank-2k symmetric matrix update
template<in-matrix InMat1, in-matrix InMat2,
        possibly-packed-inout-matrix InOutMat, class Triangle>
void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B,
InOutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2,
        possibly-packed-inout-matrix InOutMat, class Triangle>
void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                    InMat1 A, InMat2 B,
InOutMat C, Triangle t);

// rank-2k Hermitian matrix update
template<in-matrix InMat1, in-matrix InMat2,
        possibly-packed-inout-matrix InOutMat, class Triangle>
void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B,
InOutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2,
        possibly-packed-inout-matrix InOutMat, class Triangle>
void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                    InMat1 A, InMat2 B,
InOutMat C, Triangle t);

```

to read as follows.

```

// overwriting rank-2k symmetric matrix update
template<in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B,

```

```

OutMat C, Triangle t);
    template<class ExecutionPolicy,
            in-matrix InMat1, in-matrix InMat2,
            possibly-packed-out-matrix OutMat, class Triangle>
        void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
            InMat1 A, InMat2 B,
OutMat C, Triangle t);

    // updating rank-2k symmetric matrix update
    template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
            possibly-packed-out-matrix OutMat, class Triangle>
        void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);
    template<class ExecutionPolicy,
            in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
            possibly-packed-out-matrix OutMat, class Triangle>
        void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
            InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);

    // overwriting rank-2k Hermitian matrix update
    template<in-matrix InMat1, in-matrix InMat2,
            possibly-packed-out-matrix OutMat, class Triangle>
        void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B,
OutMat C, Triangle t);
    template<class ExecutionPolicy,
            in-matrix InMat1, in-matrix InMat2,
            possibly-packed-out-matrix OutMat, class Triangle>
        void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
            InMat1 A, InMat2 B,
OutMat C, Triangle t);

    // updating rank-2k Hermitian matrix update
    template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
            possibly-packed-out-matrix OutMat, class Triangle>
        void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);
    template<class ExecutionPolicy,
            in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
            possibly-packed-out-matrix OutMat, class Triangle>
        void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
            InMat1 A, InMat2 B,
InMat3 E, OutMat C, Triangle t);

```

§ 8.4 Specification of rank-k update functions

Replace the entire contents of [linalg.algs.blas3.rankk] with the following.

[Note: These functions correspond to the BLAS functions xSYRK and xHERK. – end note]

- 1 The following elements apply to all functions in [linalg.algs.blas3.rankk].
- 2 *Mandates:*

- (2.1) — If OutMat has layout_blas_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument.
- (2.2) — *possibly-multipliable*<decltype(A), decltype(transposed(A)), decltype(C)> is true.
- (2.3) — *possibly-addable*<decltype(C), decltype(E), decltype(C)> is true for those overloads that take an E parameter.

3 Preconditions:

- (3.1) — *multipliable*(A, transposed(A), C) is true. [Note: This implies that C is square. – end note]
- (3.2) — *addable*(C, E, C) is true for those overloads that take an E parameter.

4 Complexity: $O(A.\text{extent}(0) \cdot A.\text{extent}(1) \cdot A.\text{extent}(0))$.

```
template<class Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
    Scalar alpha,
    InMat A,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    Scalar alpha,
    InMat A,
    OutMat C,
    Triangle t);
```

5 Effects: Computes $C = \alpha AA^T$, where the scalar α is alpha.

```
template<in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
    InMat A,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
    ExecutionPolicy&& exec,
```

```
InMat A,
OutMat C,
Triangle t);
```

6 *Effects:* Computes $C = AA^T$.

```
template<class Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
    Scalar alpha,
    InMat A,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
         class Scalar,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    Scalar alpha,
    InMat A,
    OutMat C,
    Triangle t);
```

7 *Effects:* Computes $C = \alpha AA^H$, where the scalar α is alpha.

```
template<in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
    InMat A,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
         in-matrix InMat,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    InMat A,
    OutMat C,
    Triangle t);
```

8 *Effects:* Computes $C = AA^H$.

```
template<class Scalar,
         in-matrix InMat1,
         in-matrix InMat2,
         possibly-packed-out-matrix OutMat,
         class Triangle>
void symmetric_matrix_rank_k_update(
    Scalar alpha,
```

```

    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
        class Scalar,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void symmetric_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    Scalar alpha,
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);

```

⁹ *Effects:* Computes $C = E + \alpha AA^T$, where the scalar α is alpha.

```

template<in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void symmetric_matrix_rank_k_update(
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void symmetric_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);

```

¹⁰ *Effects:* Computes $C = E + AA^T$.

```

template<class Scalar,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_k_update(
    Scalar alpha,
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,

```

```

        class Scalar,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    Scalar alpha,
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);

```

- 11 *Effects:* Computes $C = E + \alpha AA^H$, where the scalar α is alpha.

```

template<in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_k_update(
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_k_update(
    ExecutionPolicy&& exec,
    InMat1 A,
    InMat2 E,
    OutMat C,
    Triangle t);

```

- 12 *Effects:* Computes $C = E + AA^H$.

§ 8.5 Specification of rank-2k update functions

Replace the entire contents of [linalg.algs.blas3.rank2k] with the following.

[Note: These functions correspond to the BLAS functions xSYR2K and xHER2K. – end note]

- 1 The following elements apply to all functions in [linalg.algs.blas3.rank2k].

- 2 *Mandates:*

- (2.1) — If OutMat has layout_blas_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument;
- (2.2) — *possibly-multipliable*<decltype(A), decltype(transposed(B)), decltype(C)> is true.

- (2.3) — *possibly-multiplicable*<decltype(B), decltype(transposed(A)), decltype(C)> is true.
- (2.4) — *possibly-addable*<decltype(C), decltype(E), decltype(C)> is true for those overloads that take an E parameter.

3 *Preconditions:*

- (3.1) — *multiplicable*(A, transposed(B), C) is true.
- (3.2) — *multiplicable*(B, transposed(A), C) is true. *[Note: This and the previous imply that C is square. — end note]*
- (3.3) — *addable*(C, E, C) is true for those overloads that take an E parameter.

4 *Complexity: $O(A.extent(0) \cdot A.extent(1) \cdot B.extent(0))$*

```
template<in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void symmetric_matrix_rank_2k_update(
    InMat1 A,
    InMat2 B,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void symmetric_matrix_rank_2k_update(
    ExecutionPolicy&& exec,
    InMat1 A,
    InMat2 B,
    OutMat C,
    Triangle t);
```

5 *Effects: Computes $C = AB^T + BA^T$.*

```
template<in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_2k_update(
    InMat1 A,
    InMat2 B,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        possibly-packed-out-matrix OutMat,
        class Triangle>
```

```

void hermitian_matrix_rank_2k_update(
    ExecutionPolicy&& exec,
    InMat1 A,
    InMat2 B,
    OutMat C,
    Triangle t);

```

6 *Effects:* Computes $C = AB^H + BA^H$.

```

template<in-matrix InMat1,
        in-matrix InMat2,
        in-matrix InMat3,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void symmetric_matrix_rank_2k_update(
    InMat1 A,
    InMat2 B,
    InMat3 E,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        in-matrix InMat3,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void symmetric_matrix_rank_2k_update(
    ExecutionPolicy&& exec,
    InMat1 A,
    InMat2 B,
    InMat3 E,
    OutMat C,
    Triangle t);

```

7 *Effects:* Computes $C = E + AB^T + BA^T$.

```

template<in-matrix InMat1,
        in-matrix InMat2,
        in-matrix InMat3,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_2k_update(
    InMat1 A,
    InMat2 B,
    InMat3 E,
    OutMat C,
    Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1,
        in-matrix InMat2,
        in-matrix InMat3,
        possibly-packed-out-matrix OutMat,
        class Triangle>
void hermitian_matrix_rank_2k_update(
    ExecutionPolicy&& exec,

```



```
InMat1 A,  
InMat2 B,  
InMat3 E,  
OutMat C,  
Triangle t);
```

8 *Effects*: Computes $C = E + AB^H + BA^H$.